Kix App — iPhone Duo & iOS 27.1 适配方案

文档版本：2026-10-26 | 项目：Kix Sneaker Store (SwiftUI)

一、总体适配策略

参照 DuoAdaptDemo 参考项目，采用三层适配策略：

层级	目标	核心手段
L0 — 基线合规	移除硬编码尺寸	去掉固定 padding、`maxWidth: .infinity` 按钮、固定宽高
L1 — 系统自适应	利用系统容器自动适配	`NavigationStack`/`NavigationSplitView` 替代 `NavigationView`；
L2 — Duo 增强	iOS 27.1 专属 API	`UIArrangementViewController`（通过 `UIViewControllerRepresenta

全局改动清单

1. 废弃 API 迁移：全部 8 处 NavigationView → NavigationStack（窄屏）/ NavigationSplitView（宽屏）
2. 版本门控：所有 iOS 27.1 API 使用 if #available(iOS 27.1, *) 门控
3. 内容最大宽度：所有卡片、按钮、输入框加 maxWidth 限制，防止宽屏拉伸
4. 新增文件：

- Kix/Core/Theme/AdaptiveLayout.swift — 自适应常量
- Kix/Core/Components/DuoArrangementView.swift — iOS 27.1 桥接

二、新增基础设施文件

2.1 AdaptiveLayout.swift（自适应常量）

```
// ======== 原代码 ========
// 不存在此文件，所有页面使用硬编码尺寸

// ======== 新增文件 ========
// Kix/Core/Theme/AdaptiveLayout.swift

import SwiftUI

enum AdaptiveLayout {
    /// 内容区域最大宽度，超过此值后居中显示
    static let maxContentWidth: CGFloat = 600
```

```
    /// 按钮最大宽度
    static let maxButtonWidth: CGFloat = 400

    /// 宽屏判定断点（pt）
    static let wideBreakpoint: CGFloat = 700

    /// 卡片最大宽度
    static let maxCardWidth: CGFloat = 500

    /// 根据当前尺寸类返回列数
    static func columnCount(for sizeClass: UserInterfaceSizeClass?) -> Int {
        sizeClass == .regular ? 2 : 1
    }
}

/// 自适应内容容器：窄屏全宽，宽屏限宽居中
struct AdaptiveContent<Content: View>: View {
    @Environment(\.horizontalSizeClass) private var sizeClass
    let content: () -> Content

    var body: some View {
        content()
            .frame(maxWidth: sizeClass == .regular
                   ? AdaptiveLayout.maxContentWidth
                   : .infinity)
            .frame(maxWidth: .infinity) // 居中
    }
}
```

2.2 DuoArrangementView.swift（iOS 27.1 桥接）

```
// ======== 原代码 ========
// 不存在此文件

// ======== 新增文件 ========
// Kix/Core/Components/DuoArrangementView.swift

import SwiftUI
import UIKit

/// iOS 27.1+ 使用 UIArrangementViewController 实现双栏；
/// 旧版本回退到 UISplitViewController。
struct DuoArrangementView<Primary: View, Secondary: View>: UIViewControllerRepresentable {
    let primary: Primary
    let secondary: Secondary

    init(@ViewBuilder primary: () -> Primary,
         @ViewBuilder secondary: () -> Secondary) {
        self.primary = primary()
        self.secondary = secondary()
    }

    func makeUIViewController(context: Context) -> UIViewController {
        if #available(iOS 27.1, *) {
            return makeArrangement()
        }
        return makeLegacySplit()
    }

    func updateUIViewController(_ vc: UIViewController, context: Context) {}
```

```
    @available(iOS 27.1, *)
    private func makeArrangement() -> UIViewController {
        let arrangement = UIArrangementViewController()
        let primaryVC = UIHostingController(rootView: primary)
        let secondaryVC = UIHostingController(rootView: secondary)
        let primaryNav = UINavigationController(rootViewController: primaryVC)
        let secondaryNav = UINavigationController(rootViewController: secondaryVC)
        arrangement.setViewController(primaryNav, for: .primary)
        arrangement.setViewController(secondaryNav, for: .secondary)
        arrangement.updateArrangement(.split)
        return arrangement
    }

    private func makeLegacySplit() -> UIViewController {
        let split = UISplitViewController(style: .doubleColumn)
        let primaryVC = UIHostingController(rootView: primary)
        let secondaryVC = UIHostingController(rootView: secondary)
        split.setViewController(UINavigationController(rootViewController: primaryVC),
                                for: .primary)
        split.setViewController(UINavigationController(rootViewController: secondaryVC),
                                for: .secondary)
        split.preferredDisplayMode = .oneBesideSecondary
        return split
    }
}
```

三、逐页改动对比

3.1 MainTabView — 标签栏（核心改动）

文件： Kix/Navigation/MainTabView.swift

问题： 5 个 Tab 在折叠屏宽屏下底部标签栏拉伸，每个 Tab 内容按窄屏设计。

```
// ======== 原代码 ========

struct MainTabView: View {
    @EnvironmentObject var appState: AppState
    @StateObject private var tabSelection = TabSelection()

    var body: some View {
        TabView(selection: $tabSelection.selectedTab) {
            HomeView()
                .environmentObject(appState.favoritesManager)
                .tabItem {
                    Label("首页", systemImage: tabSelection.selectedTab == 0 ? "house.fill" : "house")
                }
                .tag(0)
                .accessibilityIdentifier("tab_home")

            NotesView()
                .environmentObject(tabSelection)
                .tabItem {
```

```
                Label("笔记", systemImage: tabSelection.selectedTab == 1 ? "note.text" : "note.text")
            }
            .tag(1)
            .accessibilityIdentifier("tab_notes")

        FavoritesView()
            .environmentObject(appState.favoritesManager)
            .tabItem {
                Label("收藏", systemImage: tabSelection.selectedTab == 2 ? "heart.fill" : "heart")
            }
            .tag(2)
            .accessibilityIdentifier("tab_favorites")

        MenuView()
            .tabItem {
                Label("菜单", systemImage: tabSelection.selectedTab == 3 ? "line.3.horizontal" : "line.3.horizontal")
            }
            .tag(3)
            .accessibilityIdentifier("tab_menu")

        CartView()
            .tabItem {
                Label("购物车", systemImage: tabSelection.selectedTab == 4 ? "cart.fill" : "cart")
            }
            .tag(4)
            .accessibilityIdentifier("tab_cart")
    }
    .accentColor(.purple)
    .onAppear {
        let appearance = UITabBarAppearance()
        appearance.configureWithOpaqueBackground()
        appearance.backgroundColor = UIColor.systemBackground
        appearance.shadowColor = UIColor.black.withAlphaComponent(0.1)
        UITabBar.appearance().standardAppearance = appearance
        UITabBar.appearance().scrollEdgeAppearance = appearance
    }
  }
}

// ======== 改动后 ========

struct MainTabView: View {
  @EnvironmentObject var appState: AppState
  @StateObject private var tabSelection = TabSelection()
  @Environment(\.horizontalSizeClass) private var sizeClass

  var body: some View {
    if sizeClass == .regular {
      // 宽屏（Duo 展开态）：侧边栏导航
      NavigationSplitView {
        List(selection: $tabSelection.selectedTab) {
          Label("首页", systemImage: "house.fill").tag(0)
          Label("笔记", systemImage: "note.text").tag(1)
          Label("收藏", systemImage: "heart.fill").tag(2)
          Label("菜单", systemImage: "line.3.horizontal").tag(3)
          Label("购物车", systemImage: "cart.fill").tag(4)
        }
        .navigationTitle("KIX")
      } detail: {
        currentTabView
      }
```

```
            .tint(.purple)
        } else {
            // 窄屏（折叠态 / 普通 iPhone）：底部标签栏
            TabView(selection: $tabSelection.selectedTab) {
                HomeView()
                    .environmentObject(appState.favoritesManager)
                    .tabItem { Label("首页", systemImage: "house.fill") }
                    .tag(0).accessibilityIdentifier("tab_home")

                NotesView()
                    .environmentObject(tabSelection)
                    .tabItem { Label("笔记", systemImage: "note.text") }
                    .tag(1).accessibilityIdentifier("tab_notes")

                FavoritesView()
                    .environmentObject(appState.favoritesManager)
                    .tabItem { Label("收藏", systemImage: "heart.fill") }
                    .tag(2).accessibilityIdentifier("tab_favorites")

                MenuView()
                    .tabItem { Label("菜单", systemImage: "line.3.horizontal") }
                    .tag(3).accessibilityIdentifier("tab_menu")

                CartView()
                    .tabItem { Label("购物车", systemImage: "cart.fill") }
                    .tag(4).accessibilityIdentifier("tab_cart")
            }
            .accentColor(.purple)
            .onAppear { configureTabBarAppearance() }
        }
    }

    @ViewBuilder
    private var currentTabView: some View {
        switch tabSelection.selectedTab {
        case 0: HomeView().environmentObject(appState.favoritesManager)
        case 1: NotesView().environmentObject(tabSelection)
        case 2: FavoritesView().environmentObject(appState.favoritesManager)
        case 3: MenuView()
        case 4: CartView()
        default: HomeView().environmentObject(appState.favoritesManager)
        }
    }

    private func configureTabBarAppearance() {
        let appearance = UITabBarAppearance()
        appearance.configureWithOpaqueBackground()
        appearance.backgroundColor = UIColor.systemBackground
        appearance.shadowColor = UIColor.black.withAlphaComponent(0.1)
        UITabBar.appearance().standardAppearance = appearance
        UITabBar.appearance().scrollEdgeAppearance = appearance
    }
}
```

3.2 HomeView — 首页商品列表

文件： Kix/Features/Home/HomeView.swift

问题： 商品卡片单列全宽，宽屏下卡片过宽；NavigationView 已废弃。

改动 1：NavigationStack 替换 + 双列网格

```
// ======== 原代码 ========

var body: some View {
    NavigationView {
        ZStack {
            Color.white.ignoresSafeArea()
            VStack(alignment: .leading, spacing: 0) {
                // --- 头部 ---
                VStack(alignment: .center, spacing: 4) { ... }
                .padding(.horizontal)
                .padding(.top, 20)
                .padding(.bottom, 15)

                ScrollView {
                    // --- 商品列表 ---
                    LazyVStack(spacing: 25) {
                        ForEach(products) { product in
                            NewProductCard(product: product, ...)
                                .padding(.horizontal)
                        }
                    }
                    .padding(.top, 10)
                    .padding(.bottom, 100) // 为标签栏预留间距
                }
            }
        }
    }
}

// ======== 改动后 ========

@Environment(\.horizontalSizeClass) private var sizeClass

private var columns: [GridItem] {
    if sizeClass == .regular {
        return [GridItem(.flexible(), spacing: 20),
                GridItem(.flexible(), spacing: 20)]
    }
    return [GridItem(.flexible())]
}

var body: some View {
    NavigationStack {
        ZStack {
            Color.white.ignoresSafeArea()
            VStack(alignment: .leading, spacing: 0) {
                // --- 头部 ---
                VStack(alignment: .center, spacing: 4) { ... }
                .padding(.horizontal)
                .padding(.top, 20)
                .padding(.bottom, 15)

                ScrollView {
                    // --- 商品列表（宽屏双列） ---
                    LazyVGrid(columns: columns, spacing: 20) {
                        ForEach(products) { product in
```

```
                    NewProductCard(product: product, ...)
                        .frame(maxWidth: AdaptiveLayout.maxCardWidth)
                        .environmentObject(appState.favoritesManager)
                }
            }
            .padding(.horizontal)
            .padding(.top, 10)
            .padding(.bottom, 100)
        }
    }
  }
}
```

改动 2：NewProductCard 图片区域自适应

```
// ======== 原代码 ========

// 商品图片区域
ZStack {
  RoundedRectangle(cornerRadius: 25)
    .fill(bgGradient)
    .frame(height: 200)

  Circle()
    .fill(Color.green.opacity(0.12))
    .blur(radius: 40)
    .offset(x: 50, y: -30)

  Group {
    if let uiImage = UIImage(named: product.imageName) {
      Image(uiImage: uiImage)
        .resizable()
        .scaledToFill()
        .frame(height: 170)
        .clipped()
        .padding(.horizontal, 12)
    } else { ... }
  }
  ...
}

// ======== 改动后 ========

// 商品图片区域
ZStack {
  RoundedRectangle(cornerRadius: 25)
    .fill(bgGradient)
    .aspectRatio(1.2, contentMode: .fit)  // 自适应高度

  Circle()
    .fill(Color.green.opacity(0.12))
    .blur(radius: 40)
    .offset(x: 50, y: -30)

  Group {
    if let uiImage = UIImage(named: product.imageName) {
      Image(uiImage: uiImage)
        .resizable()
        .scaledToFill()
```

```
            .frame(maxHeight: .infinity)  // 填满自适应高度
            .clipped()
            .padding(.horizontal, 12)
        } else { ... }
    }
    ...
}
```

改动 3：新增商品 Sheet 内 NavigationView 替换

```
// ======== 原代码 ========

.sheet(isPresented: $showAddSheet) {
    NavigationView {
        Form { ... }
        .navigationTitle("新增球鞋")
    }
}

// ======== 改动后 ========

.sheet(isPresented: $showAddSheet) {
    NavigationStack {
        Form { ... }
        .navigationTitle("新增球鞋")
    }
    .presentationSizing(.form)  // 宽屏下限制表单宽度
}
```

改动 4：编辑表单 Sheet 同样替换

```
// ======== 原代码 ========

.sheet(isPresented: $showEditSheet) {
    EditProductSheet(original: product) { updated in
        onEdit(updated)
    }
}

// ======== 改动后（EditProductSheet 内部） ========

var body: some View {
    NavigationStack {  // 替换 NavigationView
        Form { ... }
        .navigationTitle("编辑球鞋")
    }
    .presentationSizing(.form)
}
```

3.3 ProductCard — 商品卡片组件

文件： Kix/Features/Home/ProductCard.swift

```
// ======== 原代码 ========

.frame(height: 180)
```

```
.cornerRadius(16)

// ======== 改动后 ========

.aspectRatio(1.0, contentMode: .fit)
.frame(maxHeight: 250)
.cornerRadius(16)
.frame(maxWidth: AdaptiveLayout.maxCardWidth)  // 限制最大宽度
```

3.4 ProductDetailView — 商品详情页

文件： Kix/Features/Home/ProductDetailView.swift

问题： 宽屏下图片过小、内容单列偏左。

```
// ======== 原代码 ========

var body: some View {
    ScrollView {
        VStack(alignment: .leading, spacing: 20) {
            Image(product.imageName)
                .resizable()
                .scaledToFit()
                .frame(height: 300)
                .cornerRadius(20)
                .padding(.top, 16)
            Text(product.name)
                .font(AppFont.title)
                .padding(.top, 8)

            // 品牌（渐变）+ 右侧价格胶囊
            HStack { ... }

            // 加入购物车按钮，右对齐
            HStack {
                Spacer()
                Button(action: { ... }) {
                    Label("加入购物车", systemImage: "cart.badge.plus")
                        ...
                }
            }

            // 描述区域
            VStack(alignment: .leading, spacing: 8) { ... }

            Spacer()
        }
        .padding(.horizontal)
        .padding(.vertical, 16)
    }
    .background(Color.backgroundPrimary)
}

// ======== 改动后 ========

@Environment(\.horizontalSizeClass) private var sizeClass

var body: some View {
```

```
    ScrollView {
        Group {
            if sizeClass == .regular {
                // 宽屏：左图右文双栏布局
                HStack(alignment: .top, spacing: 30) {
                    Image(product.imageName)
                        .resizable()
                        .scaledToFit()
                        .frame(maxHeight: 500)
                        .cornerRadius(20)
                        .frame(maxWidth: .infinity)

                    detailContent
                        .frame(maxWidth: AdaptiveLayout.maxContentWidth)
                }
                .padding()
            } else {
                // 窄屏：保持原有单列布局
                VStack(alignment: .leading, spacing: 20) {
                    Image(product.imageName)
                        .resizable()
                        .scaledToFit()
                        .frame(height: 300)
                        .cornerRadius(20)
                        .padding(.top, 16)
                    detailContent
                }
                .padding(.horizontal)
                .padding(.vertical, 16)
            }
        }
    }
    .background(Color.backgroundPrimary)
    .navigationTitle(product.name)
    .navigationBarTitleDisplayMode(.inline)
}

// 提取公共内容区域
@ViewBuilder
private var detailContent: some View {
    VStack(alignment: .leading, spacing: 20) {
        Text(product.name).font(AppFont.title).padding(.top, 8)
        // 品牌 + 价格
        HStack { ... }
        // 加入购物车按钮（居中，限宽）
        Button(action: { ... }) {
            Label("加入购物车", systemImage: "cart.badge.plus")
                .frame(maxWidth: AdaptiveLayout.maxButtonWidth)
                ...
        }
        .frame(maxWidth: .infinity)  // 按钮居中
        // 描述区域
        VStack(alignment: .leading, spacing: 8) { ... }
    }
}
```

3.5 LoginView — 登录页

文件： Kix/Features/Auth/LoginView.swift

问题： .padding(.horizontal, 35) 硬编码；按钮 maxWidth: .infinity 在宽屏过宽。

```
// ======== 原代码 ========

// 4. 带液态镜面效果的登录按钮
Button(action: { ... }) {
    Text("登录")
        .font(.system(size: 18, weight: .bold, design: .rounded))
        .kerning(2)
        .foregroundColor(.white)
        .frame(maxWidth: .infinity)    // ← 宽屏下按钮拉满全屏
        .frame(height: 60)
        .background(brandGradient)
        .cornerRadius(20)
        .shadow(color: .blue.opacity(0.4), radius: 15, y: 8)
}

...

.padding(.horizontal, 35)  // ← 硬编码 padding

// ======== 改动后 ========

// 4. 带液态镜面效果的登录按钮
Button(action: { ... }) {
    Text("登录")
        .font(.system(size: 18, weight: .bold, design: .rounded))
        .kerning(2)
        .foregroundColor(.white)
        .frame(maxWidth: AdaptiveLayout.maxButtonWidth)  // ← 限制最大宽度
        .frame(height: 60)
        .background(brandGradient)
        .cornerRadius(20)
        .shadow(color: .blue.opacity(0.4), radius: 15, y: 8)
}
.frame(maxWidth: .infinity)  // 居中

...

.padding(.horizontal)  // ← 自适应 padding
.frame(maxWidth: AdaptiveLayout.maxContentWidth)  // 整体限宽
.frame(maxWidth: .infinity)  // 居中
```

customField 辅助函数改动：

```
// ======== 原代码 ========

.padding(.horizontal, 20)
.frame(height: 60)
.background(Color(.systemGray6).opacity(0.5))
.cornerRadius(18)

// ======== 改动后 ========

.padding(.horizontal, 20)
.frame(height: 60)
.frame(maxWidth: AdaptiveLayout.maxContentWidth)  // 限制输入框宽度
.background(Color(.systemGray6).opacity(0.5))
.cornerRadius(18)
```

3.6 RegisterView — 注册页

文件： Kix/Features/Auth/RegisterView.swift

改动与 LoginView 完全一致：

```
// ======== 原代码 ========

.padding(.horizontal, 35)

// ======== 改动后 ========

.padding(.horizontal)
.frame(maxWidth: AdaptiveLayout.maxContentWidth)
.frame(maxWidth: .infinity)  // 居中

// ======== 原代码（注册按钮） ========

.frame(maxWidth: .infinity)
.frame(height: 60)

// ======== 改动后 ========

.frame(maxWidth: AdaptiveLayout.maxButtonWidth)
.frame(height: 60)
.frame(maxWidth: .infinity)  // 居中
```

3.7 FavoritesView — 收藏页

文件： Kix/Features/Favorites/FavoritesView.swift

```
// ======== 原代码 ========

var body: some View {
    NavigationView {
        VStack(alignment: .center, spacing: 0) {
            Text("收藏") ...

            if favoritesManager.favorites.isEmpty {
                ...
            } else {
                List {
                    ForEach(favoritesManager.favorites) { product in
                        HStack {
                            Image(product.imageName)
                                .resizable()
                                .scaledToFill()
                                .frame(width: 60, height: 60)
                                .clipped()
                            ...
                        }
                    }
                }
                .listStyle(PlainListStyle())
            }
        }
        .sheet(item: $selectedProduct) { product in
            NavigationView {
```

```
            ProductDetailView(product: product)
        }
    }
  }
}

// ======== 改动后 ========

@Environment(\.horizontalSizeClass) private var sizeClass

var body: some View {
  NavigationStack {
    VStack(alignment: .center, spacing: 0) {
      Text("收藏") ...

      if favoritesManager.favorites.isEmpty {
        ...
      } else {
        if sizeClass == .regular {
          // 宽屏：双列网格
          ScrollView {
            LazyVGrid(columns: [GridItem(.flexible()),
                                GridItem(.flexible())], spacing: 16) {
              ForEach(favoritesManager.favorites) { product in
                favoriteRow(product)
              }
            }
            .padding()
          }
        } else {
          List {
            ForEach(favoritesManager.favorites) { product in
              favoriteRow(product)
            }
          }
          .listStyle(PlainListStyle())
        }
      }
    }
    .sheet(item: $selectedProduct) { product in
      NavigationStack {
        ProductDetailView(product: product)
      }
      .presentationSizing(.form)
    }
  }
}

// 提取行视图复用
@ViewBuilder
private func favoriteRow(_ product: Product) -> some View {
  HStack {
    Image(product.imageName)
      .resizable()
      .scaledToFill()
      .frame(width: 60, height: 60)
      .clipped()
      .cornerRadius(8)
    ...
  }
}
```

3.8 NotesView — 笔记页

文件： Kix/Features/Notes/NotesView.swift

```
// ======== 原代码 ========

var body: some View {
    NavigationView {
        VStack(spacing: 12) {
            ...
            // 输入行
            HStack(spacing: 8) {
                TextField("快来写点什么吧...", text: $newNoteText)
                    .textFieldStyle(RoundedBorderTextFieldStyle())
                    ...
                    .padding(.horizontal)

                Button(action: addOrUpdateNote) {
                    Text(editingNote == nil ? "添加" : "更新")
                        ...
                }
                .padding(.trailing)
            }

            // 内容区域
            if notesManager.notes.isEmpty {
                ...
                .padding(.bottom, 40) // 调整视觉居中 ← 硬编码偏移
            } else {
                List { ... }
            }
        }
        .navigationTitle("")
    }
}

// ======== 改动后 ========

var body: some View {
    NavigationStack {
        VStack(spacing: 12) {
            ...
            // 输入行（限制宽度）
            HStack(spacing: 8) {
                TextField("快来写点什么吧...", text: $newNoteText)
                    .textFieldStyle(RoundedBorderTextFieldStyle())
                    ...

                Button(action: addOrUpdateNote) {
                    Text(editingNote == nil ? "添加" : "更新")
                        ...
                }
            }
            .frame(maxWidth: AdaptiveLayout.maxContentWidth)
            .frame(maxWidth: .infinity)  // 居中
            .padding(.horizontal)

            // 内容区域
            if notesManager.notes.isEmpty {
                ...
                Spacer()  // ← 替换硬编码 .padding(.bottom, 40)
```

```
        } else {
            List { ... }
        }
    }
    .navigationTitle("")
  }
}
```

3.9 MenuView — 菜单页

文件： Kix/Features/Menu/MenuView.swift

```
// ======== 原代码 ========

var body: some View {
  NavigationView {
    VStack(spacing: 0) {
      Text("菜单") ...

      Form {
        // 个人资料区域
        Section(header: Text("个人资料")) {
          NavigationLink(destination: ProfileDetailView(user: user)) {
            HStack {
              Image(systemName: "person.crop.circle")
                .resizable()
                .frame(width: 48, height: 48)  // ← 固定尺寸
              ...
            }
          }
        }
        ...
        // 退出按钮
        Section {
          Button(action: { ... }) {
            Text("退出登录")
              .frame(maxWidth: .infinity)  // ← 宽屏过宽
              .padding(12)
              ...
          }
        }
      }
    }
  }
}

// ======== 改动后 ========

var body: some View {
  NavigationStack {
    VStack(spacing: 0) {
      Text("菜单") ...

      Form {
        Section(header: Text("个人资料")) {
          NavigationLink(destination: ProfileDetailView(user: user)) {
            HStack {
              Image(systemName: "person.crop.circle")
```

```
                        .resizable()
                        .frame(width: 48, height: 48)
                    ...
                }
            }
        }
        ...
        Section {
            Button(action: { ... }) {
                Text("退出登录")
                    .frame(maxWidth: AdaptiveLayout.maxButtonWidth)  // ← 限宽
                    .frame(maxWidth: .infinity)  // 居中
                    .padding(12)
                    ...
            }
        }
    }
    .frame(maxWidth: AdaptiveLayout.maxContentWidth)  // Form 限宽
    .frame(maxWidth: .infinity)  // 居中
  }
 }
}
```

3.10 CartView — 购物车页

文件： Kix/Features/Cart/CartView.swift

```
// ======== 原代码 ========

// 结算按钮
Button(action: { showCheckoutAlert = true }) {
    Text("去结算")
        .font(.system(size: 18, weight: .bold, design: .rounded))
        .kerning(1)
        .frame(maxWidth: .infinity)  // ← 宽屏过宽
        .padding()
        ...
}

// ======== 改动后 ========

Button(action: { showCheckoutAlert = true }) {
    Text("去结算")
        .font(.system(size: 18, weight: .bold, design: .rounded))
        .kerning(1)
        .frame(maxWidth: AdaptiveLayout.maxButtonWidth)  // ← 限宽
        .frame(maxWidth: .infinity)  // 居中
        .padding()
        ...
}

// ======== 原代码（底部结算区域） ========

VStack(spacing: 12) {
    HStack {
        Text("合计：").font(.title2)
        Spacer()
        Text("$...").font(.title2).bold()
```

```
    }
    Button(action: { ... }) { ... }
}
.padding([.horizontal, .bottom])

// ======== 改动后 ========

VStack(spacing: 12) {
    HStack {
        Text("合计：").font(.title2)
        Spacer()
        Text("$...").font(.title2).bold()
    }
    Button(action: { ... }) { ... }
}
.frame(maxWidth: AdaptiveLayout.maxContentWidth)  // 限宽
.frame(maxWidth: .infinity)  // 居中
.padding([.horizontal, .bottom])

// ======== 原代码（AddTestCardSheet 内 NavigationView） ========

NavigationView {
    VStack(spacing: 20) { ... }
    .navigationTitle("添加测试卡片")
}

// ======== 改动后 ========

NavigationStack {
    VStack(spacing: 20) { ... }
    .navigationTitle("添加测试卡片")
}
.presentationSizing(.form)
```

3.11 KixLogoView — Logo 组件

文件： Kix/Core/Components/KixLogoView.swift

```
// ======== 原代码 ========

.frame(width: 1024, height: 1024)  // ← 固定 1024x1024
.clipShape(RoundedRectangle(cornerRadius: 220, style: .continuous))

// ======== 改动后 ========

.aspectRatio(1, contentMode: .fit)
.frame(maxWidth: 400)  // 自适应，最大 400pt
.clipShape(RoundedRectangle(cornerRadius: 220, style: .continuous))
```

3.12 AuthRouterView — 认证路由

文件： Kix/Features/Auth/AuthRouterView.swift

无需改动。 仅做条件判断，子视图的尺寸类环境会自动传递。

四、iOS 27.1 & iPhone Duo 适配总方案

4.1 三层架构总览

```
┌──────────────────────────────────────────────────┐
│ L2 — Duo 增强（iOS 27.1+）                       │
│ • UIArrangementViewController（primary/secondary 分栏）│
│ • reservedRegions(kind: .division / .occlusion)  │
│ • 通过 DuoArrangementView 桥接到 SwiftUI         │
├──────────────────────────────────────────────────┤
│ L1 — 系统自适应（iOS 16+）                       │
│ • NavigationSplitView / NavigationStack          │
│ • @Environment(\.horizontalSizeClass)            │
│ • LazyVGrid 自适应列数                           │
│ • AdaptiveContent 容器自动限宽居中               │
├──────────────────────────────────────────────────┤
│ L0 — 基线合规                                    │
│ • 移除 UIScreen.main 引用                        │
│ • 移除硬编码宽度/高度/padding                    │
│ • 按钮/卡片/输入框加 maxWidth 约束               │
│ • 图片使用 aspectRatio 替代固定 height           │
└──────────────────────────────────────────────────┘
```

4.2 版本兼容策略

```
// 部署目标保持 iOS 16.0
// 所有 iOS 27.1 API 调用使用版本门控：

if #available(iOS 27.1, *) {
    // 使用 UIArrangementViewController
    return AdaptiveListDetailViewController()
}
// 回退到 NavigationSplitView 或 UISplitViewController
return LegacySplitListDetailViewController()
```

4.3 各页面适配等级

页面	L0 基线	L1 自适应	L2 Duo 增强
MainTabView	无硬编码问题	TabView → NavigationSplitView	可选 Arrangement
HomeView	卡片限宽、图片自适应	单列 → 双列网格	—
ProductDetailView	按钮限宽	单栏 → 双栏（左图右文）	可选 Arrangement
LoginView	padding 自适应、按钮限宽	内容居中限宽	—
RegisterView	同 LoginView	同 LoginView	—
FavoritesView	缩略图尺寸	列表 → 双列网格	—
NotesView	输入行限宽	内容居中	—
MenuView	Form 限宽、按钮限宽	内容居中	—
CartView	按钮限宽、结算区限宽	内容居中	—
KixLogoView	去掉固定 1024x1024	aspectRatio 自适应	—

4.4 禁止事项（反模式）

禁止	原因	替代方案
`UIScreen.main.bounds`	Duo 多场景下不可靠	`GeometryReader` / `sizeClass`
固定 `.frame(width:)`	无法适配不同屏幕宽度	`.frame(maxWidth:)`
`UIDevice.current.orientation`	Duo 内屏不遵循传统方向语义	`sizeClass`
`.frame(maxWidth: .infinity)` 用于按钮	宽屏下按钮拉伸到荒谬宽度	`.frame(maxWidth: 400)`
硬编码 `.padding(.horizontal, 35)`	宽屏下内容区过宽	`.padding(.horizontal)` + 内容限宽

4.5 实施优先级

优先级	内容	预估工时
P0	新建 AdaptiveLayout.swift + DuoArrangementView.swif	1h
P0	L0：全局去掉硬编码尺寸（所有页面）	2h
P1	L1：MainTabView 改为尺寸类判断	1h
P1	L1：HomeView 双列网格	1h
P1	L1：ProductDetailView 双栏布局	1h
P1	L1：所有 NavigationView → NavigationStack	1h
P2	L1：FavoritesView / NotesView 网格化	1h
P2	L1：LoginView / RegisterView 限宽居中	0.5h
P2	L1：MenuView / CartView 限宽居中	0.5h
P3	L2：DuoArrangementView 集成测试	2h
P3	模拟器验证（折叠/展开两种状态）	1h
总计		**~12h**

五、验证清单

- □ iPhone 16（窄屏）：所有页面布局不变
- □ iPhone Duo 折叠态（~6.3"）：等同于普通 iPhone
- □ iPhone Duo 展开态（~7.6" / regular width）：
- □ MainTabView 显示侧边栏
- □ HomeView 商品双列排列
- □ ProductDetailView 左图右文
- □ LoginView / RegisterView 内容居中不拉伸
- □ 所有按钮宽度不超过 400pt
- □ 所有卡片宽度不超过 500pt
- □ iOS 27.1 真机：`reservedRegions` 正确避让铰链区域
- □ iOS 16-26 回退：`NavigationSplitView` 正常工作